

Technology Stack Report

A complete analysis of the project codebase including frontend, backend, database, authentication, APIs, animations, build tools, folder responsibilities, and architecture.

Metric	Value
Files scanned	55 authored project files
Approx lines	39,830
Architecture	React + Vite frontend, Express API backend, JSON-file persistence
Package manager	npm
Build tool	Vite

Languages Detected

Language / Format	Files	Usage	Purpose
JavaScript	32	58.2% files, 14.9% lines	Backend, course data, helpers, configs
JSX / React JavaScript	14	25.5% files, 18.6% lines	UI components and pages
CSS	1	39.9% lines	Main custom styling system
HTML	1	Vite entry	Browser shell
JSON	3	10.6% lines	Dependencies, lockfile, local database
XML / TXT / PNG	3	SEO + asset	Sitemap, robots, course image
TypeScript / Markdown / Shell	0	Not authored	Not used as project source

Core Stack

Technology	Version	Purpose	Importance
React	19.2.7	Frontend UI framework	Critical
React DOM	19.2.7	Mounts React app	Critical
Vite	8.1.0	Dev server and production build	Critical
Express.js	5.2.1	Backend API server	Critical
Node.js	Not pinned	Runtime for backend and tooling	Critical
JSON file DB	Local	Persistence through db.json	Critical
CSS	Native	Main visual design system	Critical
Framer Motion	12.42.0	Animations and transitions	High
Lucide React	1.21.0	Icon library	High
React Router DOM	7.18.0	Routing utilities	Medium

Technology	Version	Purpose	Importance
bcryptjs	3.0.3	Password hashing	High
jsonwebtoken	9.0.3	JWT auth	High
cors / dotenv	2.8.6 / 17.4.2	API support and env loading	Medium
Tailwind CSS	4.3.1	Configured; custom CSS is dominant	Low/Configured
PostCSS + Autoprefixer	8.5.15 / 10.5.2	CSS processing	Medium
React Markdown	10.1.0	Dependency installed, not actively imported	Low
concurrently	10.0.3	Runs frontend/backend together	Medium

Frontend Stack

Technology	Purpose	Used In
React + JSX	Component-based UI	src/App.jsx, src/pages/*.jsx, src/components/*.jsx
CSS	Responsive custom design system	src/styles.css
Vite	Frontend dev/build tooling	vite.config.js
Framer Motion	Animations, transitions, useInView	App.jsx, pages, LessonWidgets.jsx
Lucide React	Icons	App.jsx, pages, components
React Router DOM	Browser routing utilities	src/App.jsx
localStorage	Token, theme, notes, bookmarks, visits	App.jsx, CourseViewerPage.jsx, ThemeContext.jsx

Backend Stack

Technology	Purpose	Used In
Node.js	Backend runtime	server.js
Express.js	REST API server	server.js
CORS	Allows frontend API requests	server.js
dotenv	Loads environment variables	server.js, .env
bcryptjs	Password hashing	server.js
jsonwebtoken	JWT creation and verification	server.js
fs/path	JSON database read/write	server.js

Database Technologies

Technology	Purpose	Notes
JSON file database	Stores users, progress, quizzes, projects, discussions, admin content, leaderboard	Current active database: db.json

Technology	Purpose	Notes
MongoDB / MySQL / PostgreSQL	Course content only	Educational lesson modules, not connected runtime databases

No active Mongoose, Firebase, Prisma, SQL client, MongoDB driver, or PostgreSQL driver is installed.

Authentication System

Feature	Technology	Files
Email/password register	Express + bcryptjs + JWT	server.js
Login	bcrypt compare + JWT issue	server.js
JWT protected routes	Authorization Bearer middleware	server.js, App.jsx, ProfilePage.jsx, CourseViewerPage.jsx
Token storage	localStorage	src/App.jsx
OTP login/register	In-memory mock OTP	server.js, src/App.jsx
Google auth	Simulated endpoint	server.js, src/App.jsx
Password reset	Mock reset flow	server.js, src/App.jsx

Internal APIs

Area	Endpoints	Feature
Auth	/api/auth/register, login, send-otp, verify-otp, google, forgot-password, reset-password	Auth modal
User	/api/users/profile, change-password, streak	Profile and streak
Courses	/api/courses/complete	Lesson completion and XP
Quizzes	/api/quizzes/attempts, attempts/me	Quiz tracking
Projects	/api/projects, /api/projects/:projectId/submit	Projects and submissions
Leaderboard	/api/leaderboard	Dashboard leaderboard
Discussions	/api/discussions/:lessonId	Lesson comments
Mentor	/api/mentor/chat	Rule-based mentor chat
Admin	/api/admin/analytics, users, content/*	Admin panel

No external API provider integration was detected. Mentor chat is currently rule-based local backend logic.

Animation, UI, and State Management

Category	Technology	Purpose
Animation	Framer Motion 12.42.0	Motion components, transitions, AnimatePresence, useInView
Animation	CSS transitions/keyframes	Hover states, responsive polish, micro-interactions
UI/Icon	Lucide React 1.21.0	SVG icon components

Category	Technology	Purpose
UI	Custom components	Search, quiz, modals, lesson widgets, course cards
State	React hooks	Primary app state
State	React Context	Theme state
State	localStorage	Persistent client state
Not detected	Redux/Zustand/MobX/Recoil/TanStack Query	No dedicated state library

Build Tools and Scripts

Tool	Version	Purpose
npm	package-lock present	Package manager
Vite	8.1.0	Dev server and build
@vitejs/plugin-react	6.0.3	React support for Vite
PostCSS	8.5.15	CSS processing
Autoprefixer	10.5.2	Vendor prefixing
Tailwind CSS	4.3.1	Configured utility engine
concurrently	10.0.3	Runs frontend/backend together

```

npm scripts:
  dev: vite
  server: node server.js
  start: concurrently "node server.js" "vite"
  generate:sitemap: node scripts/generateSitemap.js
  build: npm run generate:sitemap && vite build
  preview: vite preview

```

Deployment Configuration

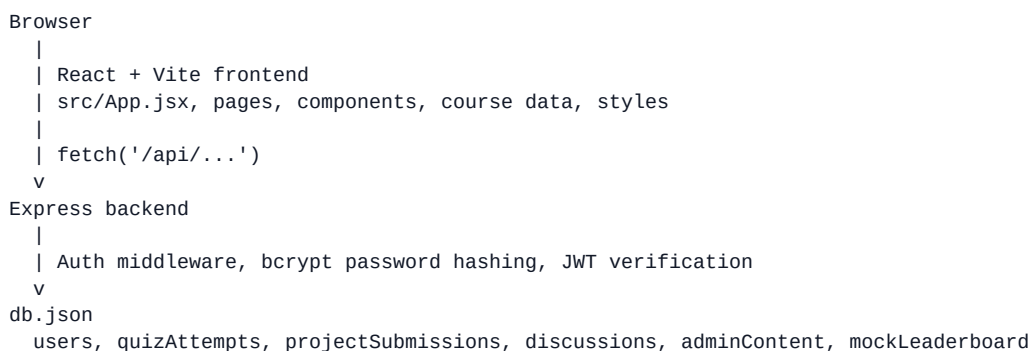
Item	Status
Production build output	dist/ generated by Vite
Sitemap generation	scripts/generateSitemap.js
Robots file	public/robots.txt
Vite proxy	/api proxied to http://localhost:5005 in dev
Vercel / Netlify / Docker / CI-CD config	Not detected

Folder Structure Responsibilities

Folder / File	Responsibility
src/	Main frontend source code
src/pages/	Route-level screens: landing, dashboard, courses, viewer, roadmap, profile
src/components/	Reusable UI blocks: search, quiz, widgets, footer modals

Folder / File	Responsibility
src/context/	React Context providers, currently theme state
src/utils/	SEO, lesson URLs, progress, validation helpers
src/courseData/	Static course/lesson content modules
src/coursesData.js	Aggregates course data
src/styles.css	Global design system and responsive CSS
public/	Static assets, robots, sitemap
scripts/	Build-time helper scripts
server.js	Express backend API
db.json	Local JSON database

Architecture Summary



Feature Dependencies

Feature	Depends On
Landing page	React, CSS, Framer Motion, Lucide, course data
Courses page	React, CSS, course data
Lesson viewer	React, CSS, Framer Motion, Lucide, localStorage, API completion endpoints
Animated lesson widgets	Framer Motion, React
Quiz system	React, QuizBlock, /api/quizzes/attempts, JWT
XP/progress	Express, JWT, db.json, /api/courses/complete
Auth modal	React, Express auth routes, bcryptjs, JWT, localStorage
Profile page	JWT, /api/users/profile, /api/users/change-password
Leaderboard	/api/leaderboard, db.json
Discussions/comments	/api/discussions/:lessonId, JWT for posting
Admin panel	JWT auth, hardcoded admin email check, /api/admin/*
SEO URLs/sitemap	src/utils/seo.js, scripts/generateSitemap.js
Theme toggle	React Context, localStorage, CSS

Improvement Priorities

Priority	Recommendation	Why
1	Course Viewer Redesign	Largest learning-experience impact
2	XP + Progress Tracking polish	Improves retention
3	Roadmap System upgrade	Career-based learning paths
4	JSON-based Content Engine	Scales better for 50+ courses
5	Fuse.js Search Upgrade	Better UX with 1000+ lessons/questions

Final Summary

Summary Item	Count / Rating
Total major technologies detected	19
Frontend technologies count	9
Backend technologies count	7
Database technologies count	1 active
Third-party runtime libraries	10
Dev/build libraries	6
Auth mechanisms	4 flows: password, JWT, OTP mock, Google mock
External API integrations	0 confirmed
Estimated project complexity	Intermediate to Advanced
Classification	Advanced frontend + intermediate backend MVP

Overall: CodeMasti is a sizeable React learning platform with rich custom UI, animated lesson experiences, static course content modules, and a lightweight Express backend for auth, progress, quizzes, projects, discussions, and admin features. The frontend is the largest and most mature part. The backend is practical for an MVP, but the JSON-file database and mock auth integrations would need upgrading for production scale.